



**Tecnológico
de Monterrey**

VANTTEC - OPEN LAB

Área de Software: Visión

Detección de objetos y estimación de profundidad

Nombre: Roberto Carlo Ponce de León Ruvalcaba

Fecha de Inicio: 02/09/2026

Investigación

Clasificación de imágenes, detección de objetos y segmentación.

La clasificación de imágenes consiste en asignar una categoría o etiqueta a una imagen, este método no identifica la ubicación del objeto ni la cantidad de instancias, determinando solamente a qué categoría pertenece la imagen globalmente. A diferencia de la clasificación, la detección de objetos no se limita a evaluar la imagen de forma global, sino que puede localizar regiones específicas y determinar el número de instancias de cada objeto presente. Esto lo logra identificando diferentes regiones dentro de la imagen de acuerdo con el nivel de confianza de su clasificación y delimitándolas mediante cajas (bounding boxes). Por último, la segmentación permite obtener una delimitación más precisa de los objetos, ya que asigna una categoría a cada píxel. La segmentación no utiliza cajas rectangulares ni clasifica globalmente la imagen, lo que permite delimitar la forma o máscara exacta del objeto sin incluir los elementos del fondo.

Bounding box y qué información contiene.

Una bounding box (caja delimitadora) es un rectángulo alineado con los ejes de la imagen que encierra de forma ajustada la silueta de un objeto detectado. Es la representación geométrica estándar utilizada en detección de objetos para delimitar la región espacial de interés sin tener que procesar la forma exacta píxel por píxel. La información que almacena son las coordenadas que definen la ubicación y el tamaño del rectángulo dentro de la imagen. Esto se representa comúnmente mediante uno de los dos siguientes formatos: guardando las esquinas opuestas (la esquina superior izquierda y la esquina inferior derecha) o registrando las coordenadas del centro del objeto (u,v) junto con su ancho (w) y su altura (h). También contiene la categoría asignada al objeto y el nivel de confianza, que es un valor normalmente expresado en un rango de 0 a 1 que representa el nivel de confianza del modelo en la detección realizada de ese objeto.

Las coordenadas (u,v) dentro de una imagen digital.

Dentro de una imagen digital, las coordenadas (u, v) representan la ubicación exacta de un píxel dentro de la imagen. Por convención, el origen (0,0) se ubica en la esquina superior izquierda de la imagen: el eje ' u ' corresponde al eje horizontal (columnas) y aumenta hacia la derecha, mientras que el eje ' v ' corresponde al eje vertical (filas) y aumenta hacia abajo.

El valor de confianza producido por un modelo de detección.

El valor de confianza es un número, expresado normalmente en un rango de 0 a 1, que mide el grado de certeza o seguridad que tiene el modelo sobre su propia predicción. Normalmente, este valor se utiliza para filtrar las detecciones erróneas y los falsos positivos, estableciendo un umbral mínimo de confianza (threshold) para que el sistema procese y poder visualizar en las que el modelo tenga un valor alto de certeza sobre la presencia del objeto.

Qué métodos existen para detectar objetos y cuál utilizarás.

Los métodos para detectar objetos se dividen fundamentalmente en tres categorías. Por un lado, están los métodos clásicos de visión por computadora, los cuales utilizan características extraídas manualmente, como colores, bordes, formas o propiedades geométricas, para identificar objetos. Por otro lado, se encuentran los algoritmos de aprendizaje profundo, que pueden utilizar arquitecturas de dos etapas o de una sola etapa para detectar y clasificar objetos automáticamente. Finalmente, existen los métodos Transformer, los cuales utilizan mecanismos de atención para analizar globalmente una imagen, detectar los elementos presentes en una escena y comprender las relaciones y el contexto entre ellos.

Para este proyecto se utilizará un método clásico de visión por computadora basado en el análisis de color y las propiedades geométricas del objeto. Esta opción fue seleccionada principalmente por el tiempo disponible para desarrollar el proyecto y por mi experiencia previa trabajando con herramientas y técnicas similares de procesamiento de imágenes. Con este enfoque se busca avanzar de manera eficiente y poder cumplir con los objetivos principales de la actividad.

La profundidad en una imagen y qué métodos existen para estimarla.

La profundidad en una imagen es la distancia física real desde el centro óptico de la cámara hasta los objetos presentes en la escena. Dado que una cámara convencional convierte un entorno tridimensional en una matriz bidimensional definida únicamente por coordenadas de píxeles (u, v), la dimensión de la distancia se pierde durante la captura. La estimación de profundidad es el proceso computacional para recuperar o inferir esa dimensión. Para hacer su estimación, existen principalmente tres métodos: Estimación de profundidad monocular (uso de una sola cámara). Visión estéreo (uso de dos cámaras en configuración par). Sensores activos de profundidad (uso de hardware especializado como cámaras RGB-D).

Diferencias entre profundidad monocular, visión estéreo y cámaras con sensores de profundidad.

El objetivo es conocer la distancia entre la cámara y un determinado objeto. Para ello, existen diferentes formas de realizarlo, dependiendo del hardware disponible y de los requerimientos de cada proyecto. Una de estas formas es la profundidad monocular, que utiliza únicamente una cámara estándar y se estima la distancia de un objeto analizando patrones contextuales en la imagen (como el tamaño relativo de los objetos, la perspectiva, las sombras y la oclusión), una de sus desventajas es que suele entregar una profundidad relativa en lugar de una distancia métrica.

La visión estéreo se basa en la visión binocular de los humanos, utilizando dos cámaras alineadas a una distancia fija (baseline). Este método compara la diferencia de posición de un mismo punto entre la toma izquierda y la derecha (disparidad) para calcular la distancia métrica mediante triangulación geométrica. Su principal desventaja es la carga computacional que requiere el procesamiento simultáneo de ambas y que necesita una calibración constante. Las cámaras con sensores de profundidad (RGB-D) tienen un hardware especializado que emite señales de luz no visible, como con los sensores ToF. Estos sensores miden directamente el tiempo que tarda la luz infrarroja en reflejarse sobre un objeto y regresar al detector, lo que nos permite obtener una distancia métrica precisa para cada píxel. Sin embargo, este tipo de hardware especializado suele tener un costo más elevado y su rendimiento puede verse afectado por factores del entorno y presentar cierto nivel de ruido en las mediciones.

Cuáles son las limitaciones de estimar profundidad utilizando una sola webcam.

La estimación de profundidad utilizando una sola webcam presenta varias limitaciones, ya que la cámara solo puede capturar información de forma bidimensional de la escena y la profundidad debe inferirse mediante algoritmos de visión por computadora. Esto puede generar errores en la estimación de la distancia, especialmente cuando los objetos tienen pocas características visuales, presentan superficies uniformes, transparentes o muy reflectantes. Además, factores del entorno, como la iluminación, las sombras y los cambios en la escena, pueden afectar los resultados. Otra limitación es la dificultad para obtener una distancia métrica absoluta, debido a que una sola imagen no contiene suficiente información para determinar directamente la escala y la profundidad real de los objetos. Por esta razón, pueden ser necesarias técnicas de calibración, referencias conocidas o modelos especializados para mejorar la precisión de la estimación.

Si el método seleccionado produce profundidad relativa o distancia métrica.

El método seleccionado permite obtener una estimación de la distancia métrica, ya que utiliza el tamaño conocido del objeto y sus dimensiones aparentes en la imagen, junto con los parámetros obtenidos mediante la calibración de la cámara. Cabe aclarar que se trata de una estimación y no de una medición directa de profundidad, por lo que su precisión depende de la calibración, el tamaño del objeto y las condiciones de observación.

Qué modelos, algoritmos y librerías utilizarás y por qué los seleccionaste.

La implementación se realizará en C++ utilizando OpenCV, ya que cuenta con las herramientas necesarias para realizar el procesamiento de imágenes y la detección del objeto. También se utilizarán librerías estándar de C++. `<cmath>` se utilizará para las operaciones matemáticas necesarias para calcular la distancia y `<vector>` para almacenar los contornos detectados del objeto. Además, se utilizará un archivo de encabezado propio, llamado *Vision.hpp*, para organizar y declarar las funciones relacionadas con el sistema de visión de la cámara.

- La razón por la que se utilizaron estos métodos y herramientas es para realizar el siguiente proceso:

Primero, la imagen obtenida por la cámara se convierte de BGR a HSV. Este se seleccionó porque permite separar mejor la tonalidad, saturación y brillo de los colores, facilitando la identificación de la boya por su color rojo. Posteriormente, se realiza una umbralización por rangos de color para obtener los píxeles correspondientes al objeto. Se utilizan dos rangos debido a que el rojo se encuentra en ambos extremos del rango de tonalidad (Hue) de HSV. Después se aplican filtros para reducir el ruido y mejorar la región detectada. Posteriormente, se realiza una detección de contornos para identificar las regiones de la máscara y trabajar con sus propiedades geométricas de las regiones detectadas. De los contornos encontrados se selecciona el de mayor área y se verifica que supere un área mínima para descartar posibles detecciones de ruido. Finalmente, se utilizan las propiedades geométricas de la boya y de la cámara para estimar su posición. Debido a que se conoce el diámetro real de la boya, se relaciona con su diámetro aparente en píxeles para calcular la distancia en el eje Z. El parámetro focal utilizado se obtuvo mediante una calibración experimental con diferentes distancias conocidas entre la cámara y la boya, con el objetivo de obtener un valor adecuado para la cámara utilizada. Considerando la posición de la boya respecto al centro de la imagen, se calculan los desplazamientos en los ejes X y Y. Con estos valores y la distancia Z se obtiene la distancia tridimensional entre la cámara y la boya. En conjunto, estos algoritmos y librerías fueron seleccionados porque son adecuados para las características de la boya y permiten realizar el procesamiento de manera eficiente.

Docker y la diferencia entre una imagen, un contenedor y un Dockerfile.

El funcionamiento de Docker se basa en compartir el kernel del sistema operativo anfitrión para ejecutar entornos aislados sin utilizar una máquina virtual. Docker utiliza funcionalidades nativas del kernel de Linux. Esto permite que aplicaciones se ejecuten de manera independiente, segura y con un consumo de recursos mínimo. El Dockerfile es un archivo de texto con las instrucciones secuenciales necesarias para construir el entorno. A partir de este archivo se compila la imagen, que es una plantilla ejecutable e inmutable de solo lectura que contiene todo lo necesario, desde el código fuente hasta las librerías y dependencias. Por último, el contenedor es la instancia en ejecución de una imagen; añade una capa de lectura y escritura para operar en tiempo real y representa el entorno donde corre la aplicación.

Proporcionar acceso a una webcam desde un contenedor Docker.

Para utilizar una cámara desde un contenedor Docker en Linux, se proporciona acceso al dispositivo de video mediante el parámetro `--device`. En nuestro caso, la Raspberry Pi reconoce la cámara como `/dev/video0`, por lo que se utiliza `--device=/dev/video0:/dev/video0` para permitir que el programa de visión desarrollado con OpenCV acceda a ella y capture imágenes en tiempo real. Este acceso se configuró directamente en los scripts `run.sh` y `runCalibration.sh`.

Bibliografía:

IBM. (s. f.). *Clasificación de imágenes*. IBM.

<https://www.ibm.com/mx-es/think/topics/image-classification>

IBM. (s. f.). *Detección de objetos*. IBM.

<https://www.ibm.com/mx-es/think/topics/object-detection>

IBM. (s. f.). *Segmentación de instancias*. IBM.

<https://www.ibm.com/mx-es/think/topics/instance-segmentation>

OpenCV. (s. f.). *Image processing*. OpenCV.

https://docs.opencv.org/5.0/main_modules/imgproc.html

Ultralytics. (s. f.). *Bounding box*. Ultralytics.

<https://www.ultralytics.com/es/glossary/bounding-box>

Ultralytics. (s. f.). *Confidence*. Ultralytics.

<https://www.ultralytics.com/es/glossary/confidence>

Ultralytics. (s. f.). *One-stage object detectors*. Ultralytics.

<https://www.ultralytics.com/es/glossary/one-stage-object-detectors>

Ultralytics. (s. f.). *Two-stage object detectors*. Ultralytics.

<https://www.ultralytics.com/es/glossary/two-stage-object-detectors>

Ultralytics. (s. f.). *Transformer*. Ultralytics.

<https://www.ultralytics.com/es/glossary/transformer>

Ultralytics. (s. f.). *What is monocular depth estimation? An overview*. Ultralytics.

<https://www.ultralytics.com/es/blog/what-is-monocular-depth-estimation-an-overview>

Red Hat. (s. f.). *¿Qué es Docker?* Red Hat.

<https://www.redhat.com/es/topics/containers/what-is-docker>